Agreement Estimation

Data Science with Applications in Natural Language Processing

Goals

By the end, we will be able to:

- **measure** the percent agreement of two annotators
- **explain** why chance agreement inflates it
- **compute** Cohen's κ by hand and with sklearn
- **interpret** a κ value, and its limits
- **choose** a measure for many annotators (Fleiss, Krippendorff)
- **diagnose** where and why annotators disagree on real data

Outline

1. Percent agreement
2. Chance agreement and Cohen's κ
3. Interpreting κ
4. Many annotators
5. Application: themes of Greek folk songs

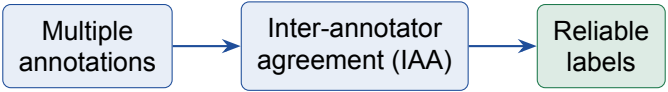
A question

Two annotators agree 82% of the time.

Is that good?

- What if 90% of the data is “neutral”?
- What if the annotators answered **at random**?

Why measure agreement?



- **High**: clear guidelines, well-defined scheme
- **Low**: vague guidelines, subjectivity, ambiguity

Agreement is also a ceiling

- A model learns from human labels
- If humans agree 70% of the time, what does 95% accuracy mean?
- Annotator agreement gives a **realistic upper bound**

More on this topic: evaluating classification models.

Percent agreement

$$P_o = \frac{\text{items where they agree}}{\text{all items}}$$

- 82 agreements in 100 messages $\Rightarrow P_o = 0.82$
- o = **observed** agreement

Two annotators flipping coins

```
import numpy as np, pandas as pd
np.random.seed(2026)
annots = pd.DataFrame({"A1": np.random.randint(0, 2, 100),
                       "A2": np.random.randint(0, 2, 100)})
annots["agree"] = annots.A1 == annots.A2
annots.sample(3, random_state=2026)
```

	A1	A2	agree
98	0	1	False
10	1	1	True
4	1	0	False

The mean of the Trues

```
po = annots.agree.mean()
print(f"Po: {po:.2f}")
# Po: 0.50
```

- True = 1, False = 0 $\Rightarrow$ the mean is a proportion
- The annotators answered **at random**, yet they agree 50% of the time!

Chance agreement

- A1 says “1” in **90%** of cases
- A2 says “1” in **50%**
- If they answer independently:

both “1”	0.9×0.5	$= 0.45$
both “0”	0.1×0.5	$= 0.05$
chance agreement P_e		**0.50**

e = expected: multiply per category, then add.

The more skewed the labels, the “easier” the agreement

P(A1 says 1)	P(A2 says 1)	P_e
0.5	0.5	0.50
0.9	0.5	0.50
0.9	0.9	**0.82**
0.99	0.99	**0.98**

- Two annotators who almost always say “neutral” agree **without reading**

Step 1: relative frequencies of pairs

```
ncounts = pd.crosstab(annots.A1, annots.A2, normalize=True)
```

	A2 = 0	A2 = 1	*A1*
A1 = 0	**0.23**	0.25	0.48
A1 = 1	0.25	**0.27**	0.52
A2	0.48	0.52	

- Diagonal = agreement: $0.23 + 0.27 = 0.50$
- Margins = how often each annotator uses each label

Step 2: P_e from the margins

```
p1 = ncounts.sum(axis=1)    # A1: 0 -> 0.48, 1 -> 0.52
p2 = ncounts.sum(axis=0)    # A2: 0 -> 0.48, 1 -> 0.52
pe = (p1 * p2).sum()
print(f"{pe:.4f}")
# 0.5008  (= 0.48*0.48 + 0.52*0.52)
```

- Index alignment: the same categories get multiplied

We saw this with tables: operations align on the index first.

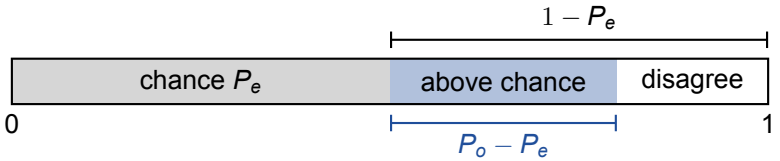
Cohen's κ

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

- Numerator: how far **above chance** we agree
- Denominator: how far above chance we **could** agree
- $P_o = 1 \Rightarrow \kappa = 1 \qquad P_o = P_e \Rightarrow \kappa = 0$

Cohen (1960). A coefficient of agreement for nominal scales.

A picture



κ = the blue part as a fraction of $1 - P_e$

Step 3: κ

```
k = (po - pe) / (1 - pe)
print(f"{k:.2f}")      # -0.00
print(f"{k:g}")        # -0.00160256
```

- $\kappa \approx 0$: the 50% agreement was **all chance**
- Makes sense: the “annotators” were random number generators

In practice: sklearn

```
from sklearn.metrics import cohen_kappa_score
cohen_kappa_score(annots.A1, annots.A2)
# -0.0016025641025640969
```

- Same result as the three steps
- Works with **string labels** too, not only 0/1
- The order of the annotators does not matter

Remember the 80% agreement?

From the sentiment annotation (positive / negative / neutral):

```
ann_a = ["pos","neg","pos","neu","neg","pos","neu","neg","pos","neu"]
ann_b = ["pos","neg","neu","neu","neg","pos","neg","neg","pos","neu"]
cohen_kappa_score(ann_a, ann_b)      # 0.70
```

- $P_o = 0.80, \quad P_e = 0.4 \cdot 0.3 + 0.3 \cdot 0.4 + 0.3 \cdot 0.3 = 0.33$
- $\kappa = (0.80 - 0.33)/(1 - 0.33) = 0.70$

Three categories: three products in P_e.

Negative κ

$\kappa < 0$: less agreement than chance

- **Systematic** disagreement: one says “positive” whenever the other says “negative”
- Or one category dominates and P_e is very large
- Usually means: look at **the data**, not only the number

How many decimals?

Quantity	Example	f-string
κ, α	0.63	:.2f
percentage	93.4%	:.1%
p-value	< 0.001	:.3f

- 0.628374 is not “more accurate” than 0.63
- It is **false precision**: noise for the reader

The Landis & Koch scale (1977)

κ	Agreement
≤ 0	poor (no better than chance)
0.00 – 0.20	slight
0.21 – 0.40	fair
0.41 – 0.60	moderate
0.61 – 0.80	substantial
0.81 – 1.00	almost perfect

A scale, not a law

- The thresholds are a **convention**, not a statistical test
- Subjective tasks (irony, emotion): 0.5 may be good
- Objective tasks (language of a text): 0.5 is a problem
- Report **both P_o and** κ, and the number of items

The κ “paradox”

100 messages, almost all “neutral”:

```
a = ["neu"]*81 + ["neu"]*9 + ["oth"]*9 + ["oth"]*1
b = ["neu"]*81 + ["oth"]*9 + ["neu"]*9 + ["oth"]*1
```

- $P_o = 0.82$ but $P_e = 0.9 \cdot 0.9 + 0.1 \cdot 0.1 = 0.82$
- $\kappa = \mathbf{0.00}$

$\Rightarrow$ High agreement, zero κ: they agreed on **none** of the rare cases

That is why the 82% at the start says nothing on its own.

More than two annotators

Mean pairwise κ

- κ for each pair
- average

Fleiss' κ

- one measure for all
- same number of annotators per item

Krippendorff's α

- missing values
- different annotators per item
- nominal, ordinal, numeric

Three annotators, ten items

```
# rows = annotators, columns = items
data = np.array([[0,1,2,2,1,0,1,2,0,1],
                 [0,2,2,2,1,0,1,1,0,1],
                 [0,1,2,1,1,0,1,2,0,1]])
```

```
from itertools import combinations
ks = [cohen_kappa_score(data[i], data[j])
      for i, j in combinations(range(3), 2)]
# [0.70, 0.85, 0.54] -> mean 0.69
```

Fleiss' κ on the same data (statsmodels): 0.69.

Krippendorff's α

```
import krippendorff          # pip install krippendorff
krippendorff.alpha(reliability_data=data)
# 0.829
```

- 0.83 when the mean κ was 0.69? Something is off…

Pitfall: the level of measurement

```
for level in ["nominal", "ordinal", "interval"]:
    a = krippendorff.alpha(reliability_data=data,
                           level_of_measurement=level)
```

level	1 vs 2 are…	α
nominal	just different	**0.70**
ordinal	neighbours in order	0.84
interval (default)	distance 1	0.83

- For **categories** (topic, emotion): "nominal"

Missing annotations

```
d = data.astype(float)
d[2, 3] = np.nan          # annotator 3 skipped item 4
d[1, 7] = np.nan          # annotator 2 skipped item 8
krippendorff.alpha(reliability_data=d,
                   level_of_measurement="nominal")   # 0.90
```

- α uses whatever is there
- Here it went up, because exactly two **disagreements** are missing
- Missingness is not neutral: which items get skipped?

The experiment

- 100 Greek folk songs: 50 *lianotragouda* (couplets), 50 others
- 2 annotators, 30–40 minutes each
- Per song:
  - **one** main theme (15 options)
  - optionally, a suggested new category
  - **certainty** 0–5
- Platform: a spreadsheet with a drop-down list

The categories

- LOVE / ROMANCE / FLIRT
- ROMANTIC DISAPPOINTMENT
- BETRAYAL / JEALOUSY
- ABDUCTION / BRIDE KIDNAPPING
- WEDDING / BRIDAL
- FAMILY / KINSHIP
- EXILE / EMIGRATION
- EMOTIONAL PAIN / SUFFERING
- DEATH / LAMENTS
- HEROIC / KLEPHTIC
- WAR / BATTLES
- HISTORICAL EVENTS
- BALLADS / NARRATIVE
- SOCIAL / SATIRICAL
- OTHER / UNSPECIFIED

Translated from Greek. Which pairs do you expect to be confused?

From the guidelines

- **Dominance rule**: the theme that drives the narrative, not secondary motifs
- Always **one** category, even if it is “OTHER”
- “OTHER” only if:
  - the text is a fragment
  - no theme dominates
  - no category covers it
- Two worked examples (a couplet and a lament)

An example from the guidelines

Τί ὡραιότης εἶν' αὐτή, τί κάλλη καὶ τί νοῦρι;
τ' ἀγγελικό σου τὸ κορμὶ δὲν ἔχει πλιὸ κουσσοῦρι.

What beauty is this, what charm and grace? / your angelic body has no flaw at all.

⇓

LOVE / ROMANCE / FLIRT

A couplet: two lines, often no context, so hard to annotate.

Joining the two sheets: the fragile way

```
a1 = pd.read_excel("ann3.xlsx")
a2 = pd.read_excel("ann4.xlsx")
data = pd.DataFrame()
data["title"] = a1.title
data["judge1"] = a1.categories
data["judge2"] = a2.categories    # by position!
```

- Works **only** if the rows are in the same order
- Did an annotator sort their sheet? Silently wrong result
- Here the order happened to match. Next time?

Joining on a key

```
cols = ["title", "categories", "certainty"]
data = a1[cols].merge(a2[cols + ["group"]], on="title",
                      suffixes=("_1", "_2"),
                      validate="one_to_one")
data.shape       # (100, 6)
```

- Join on the **title**, not on the position
- validate stops if any title is duplicated

We saw merge in data processing.

Category distributions

```
data[["categories_1", "categories_2"]].apply(pd.Series.value_counts)
```

	Annotator 1	Annotator 2
LOVE / ROMANCE / FLIRT	25	25
HEROIC / KLEPHTIC	17	20
ROMANTIC DISAPPOINTMENT	9	13
OTHER / UNSPECIFIED	**10**	**0**
WAR / BATTLES	**7**	**1**
WEDDING / BRIDAL	3	7
…		
categories used	15	12

Where an annotator never used a category, value_counts gives NaN.

Certainty

	min	median	max	std
Annotator 1	**0**	4	5	2.0
Annotator 2	2	3	5	1.0

- Annotator 1 uses the whole scale, annotator 2 only 2–5
- Annotator 1 also has one **empty** certainty (NaN)
- Less certain, or just **calibrating** the scale differently?

Uncertainty → “OTHER”?

```
low = data[data.certainty_1 < 3]
low.categories_1.value_counts().head(3)
```

OTHER / UNSPECIFIED	**7**
LOVE / ROMANCE / FLIRT	5
WAR / BATTLES	4

- 7 of annotator 1’s 10 “OTHER” are at low certainty
- For those 10 “OTHER”, annotator 2 chose 8 **different** categories

The agreement

```
c1, c2 = data.categories_1, data.categories_2
po = (c1 == c2).mean()                # 0.600
k  = cohen_kappa_score(c1, c2)        # 0.542
```

- $P_o = 0.60$, $\kappa = 0.54$: **moderate** agreement
- With 15 categories P_e is small (0.13), so κ is close to P_o

Agreement by certainty

```
sure = (data.certainty_1 >= 3) & (data.certainty_2 >= 3)
cohen_kappa_score(c1[sure], c2[sure])
```

Filter	songs	κ	
all	100%	0.54	moderate
both ≥ 3	62%	0.62	substantial
both > 3	29%	**0.86**	almost perfect

A NaN certainty compares as False: that song is excluded.

What does this mean?

- Disagreement concentrates on the **ambiguous** songs
- Certainty is a useful signal: always ask for it
- Careful: keeping only the “easy” items inflates agreement
  - a model will see the hard ones too
  - report κ on **all** the data as well

Where do they disagree?

Annotator 1	Annotator 2	times
LOVE / ROMANCE	ROMANTIC DISAPPOINTMENT	4
WAR / BATTLES	DEATH / LAMENTS	3
WAR / BATTLES	HEROIC / KLEPHTIC	3
BETRAYAL / JEALOUSY	ROMANTIC DISAPPOINTMENT	2
HEROIC / KLEPHTIC	HISTORICAL EVENTS	2

- **Neighbouring** categories, not random errors
- Annotator 2 almost never uses WAR

pd.crosstab(c1, c2), keeping the off-diagonal cells.

Couplets or others?

Group	songs	P_o	κ
couplets	50	0.64	0.51
others	50	0.56	0.47

- A small difference with 50 songs per group: **don't** draw conclusions
- How much would κ change with another 50 songs?

data.groupby("group") and κ per group.

From measurement to action

- **Merge** or better define neighbouring categories
  - LOVE vs ROMANTIC DISAPPOINTMENT: when is it which?
- Add examples for the **hard** pairs
- Discuss disagreements with the annotators, then measure again
- Decide what to do with ambiguous items: a third annotator? multiple labels?

Summary

- P_o: how often they agree, but it includes **chance**
- $\kappa = (P_o - P_e)/(1 - P_e)$: agreement above chance
- Interpret with a scale, but with **judgement** about the task
- Many annotators: mean κ, Fleiss, Krippendorff (nominal!)
- Join annotations on a **key**, not by position
- The number is the start: then look at **where** they disagree

Exercise 1: by hand

Two annotators, 20 film reviews (positive / negative):

	B: pos.	B: neg.
A: pos.	12	2
A: neg.	1	5

Compute P_o, P_e and κ. Check with sklearn.

Exercise 1: solution

- $P_o = (12 + 5)/20 = 0.85$
- A: pos. 14/20 = 0.70 B: pos. 13/20 = 0.65
- $P_e = 0.70 \cdot 0.65 + 0.30 \cdot 0.35 = 0.455 + 0.105 = 0.56$
- $\kappa = (0.85 - 0.56)/(1 - 0.56) = 0.66$ substantial

```
a = ["p"]*12 + ["p"]*2 + ["n"]*1 + ["n"]*5
b = ["p"]*12 + ["n"]*2 + ["p"]*1 + ["n"]*5
cohen_kappa_score(a, b)     # 0.659
```

Exercise 2: the folk songs

1. Join the two sheets with merge on the title
2. Compute κ with “OTHER” **removed** (songs where either annotator used it)
3. Merge LOVE and ROMANTIC DISAPPOINTMENT into one category. How much does κ change?
4. Compute Krippendorff’s α (nominal) on the categories

For α: encode the categories as integers (pd.factorize).

Exercise 2: solution

Version	songs	P_o	κ
all	100	0.60	0.54
without “OTHER”	90	0.67	0.61
LOVE + ROM. DISAPPOINTMENT	100	0.65	0.57

- Krippendorff’s α (nominal) = 0.54, almost equal to κ
- Every “fix” raises κ, but it **changes the task**

For discussion

1. κ is 0.86 on only 29% of the data. What do you report in the paper?
2. Two annotators use the certainty scale differently. How do you compare them?
3. In which task would $\kappa = 0.4$ be satisfactory?
4. If annotators disagree systematically, who is “right”?